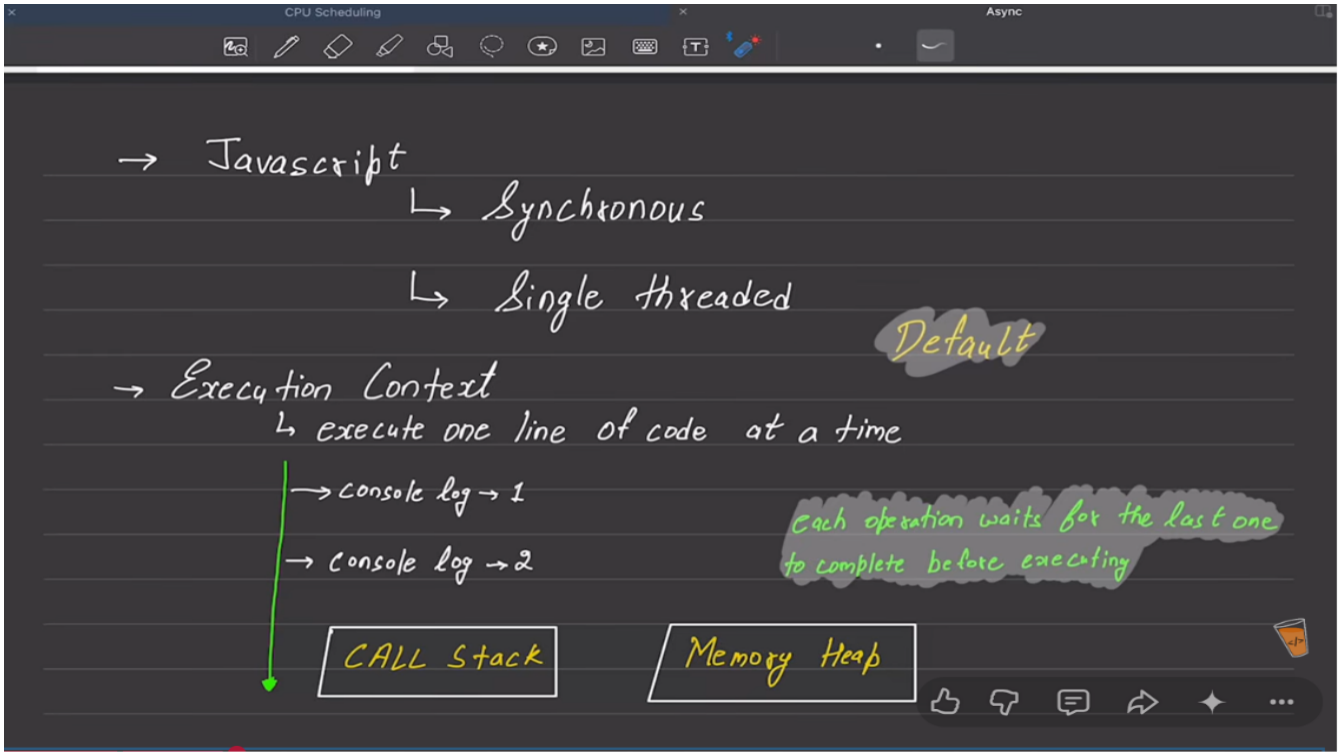
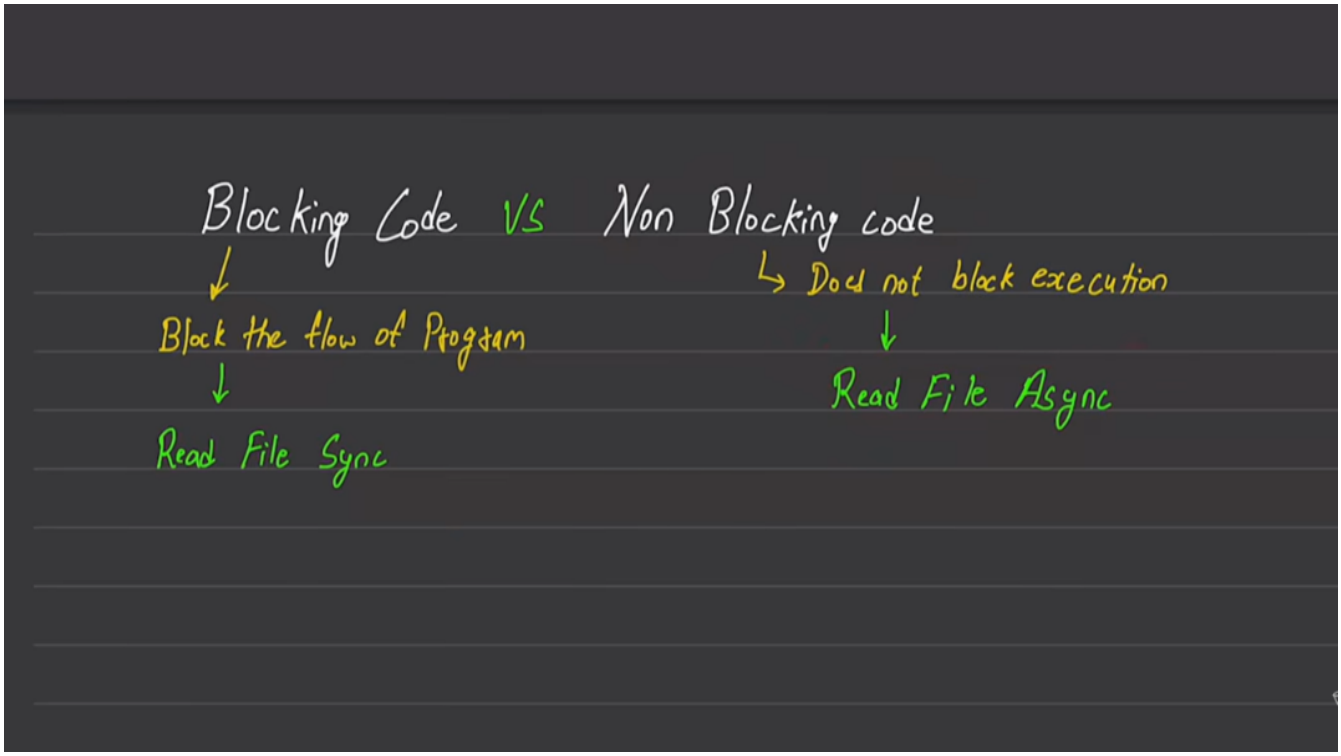
JavaScript – Synchronous, Single Threaded & Async Notes

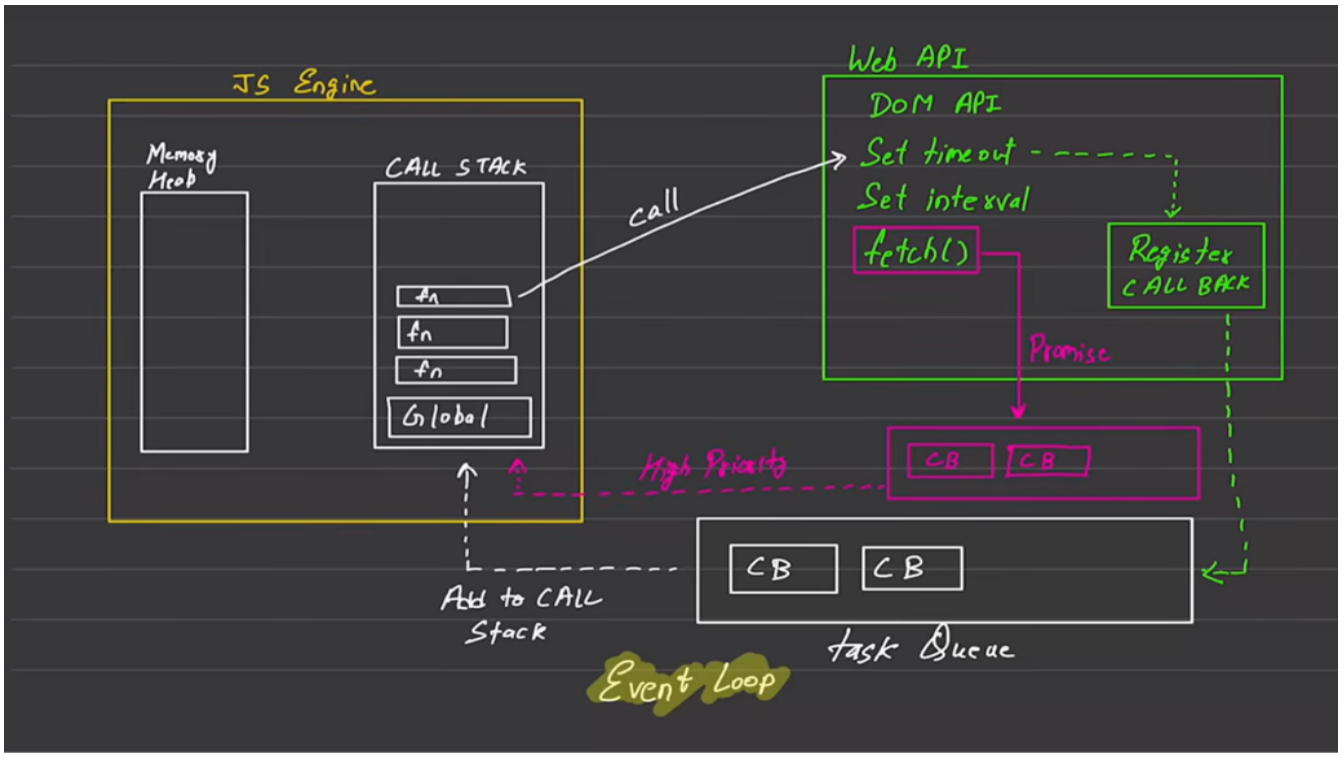
These notes summarize JavaScript execution, synchronous behavior, blocking vs non-blocking code, and the event loop. Screenshots from your lecture are included for reference.



JavaScript is synchronous and single-threaded; execution context runs one line at a time.



Blocking code vs Non-blocking code.



JavaScript Engine, Web APIs, Event Loop, Task Queue & Microtask Queue.

Detailed Notes

1. JavaScript Nature
- JavaScript is synchronous by default.
- Executes code line by line.
- Single-threaded: one task at a time.

2. Execution Context
• JavaScript executes one line of code at a time.
• Each operation waits for the previous one to complete.

3. JavaScript Engine Components
Call Stack:
• Tracks function execution.
• Uses LIFO (Last In First Out).

Memory Heap:
• Stores variables and objects.

4. Blocking vs Non-Blocking Code
Blocking Code:
• Stops the flow of the program.
• Example: readFileSync()

Non-Blocking Code:
• Does not block execution.
• Example: readFile()

5. Async JavaScript
Handled using:
• Web APIs (DOM, setTimeout, setInterval, fetch)
• Callback Queue (Task Queue)
• Microtask Queue (Promises, fetch)
• Event Loop

Important:
• Microtask Queue has higher priority than Callback Queue.

6. Event Loop
Checks if Call Stack is empty.
Execution order:
1. Microtask Queue
2. Callback Queue

Example Output:
Start
End
Promise
Timer

7. Interview Points
• JavaScript is synchronous by default.
• JavaScript is single-threaded.
• Promise executes before setTimeout().
• Event Loop manages async execution.